HTTP Analysis Using Wireshark - Text Traffic

OJE DOMINION MAYOWA

FWSD2511507

Computer and Digital Forensics

MR. Aminu Idris

8 September 2026

LAB OVERVIEW

This lab focused on generating, capturing, and forensically analyzing HTTP network traffic on a Kali Linux VM to understand how web communication works at the packet level, and how to preserve that evidence properly for investigative purposes.

I began by setting up a simple web page **(basic.html)** served locally through Apache2, then verified it was reachable and responding correctly using both a browser and `curl`. Once the web service was confirmed working, I used tshark to capture live traffic on the loopback interface while generating an HTTP request with curl, producing a .pcapng capture file containing the full client-server exchange. To maintain evidence integrity, I created a working copy of the capture and generated SHA-256 hashes for both the original and the copy, confirming no data had been altered during analysis, a core principle of digital forensic chain-of-custody.

From there, I analyzed the capture in detail: identifying the TCP three-way handshake (SYN, SYN-ACK, ACK), extracting the HTTP request and response headers (GET method, URI, Host, User-Agent, status code, server, content type), reconstructing the full request/response exchange, and tracing how sequence and acknowledgement numbers advanced in exact step with each packet's payload length. I also examined how data is encapsulated across the Ethernet, IP, TCP, and HTTP layers, and observed how the connection was gracefully closed through a FIN/ACK exchange rather than an abrupt reset. Finally, I noted that since the capture was taken over loopback, no genuine Ethernet MAC addresses were visible, since loopback traffic never actually touches a physical or virtual network interface.

OBJECTIVES

- Explain the forensic value of HTTP, TCP, IP and Ethernet metadata.
- Create and serve a simple HTML page using Apache.
- Capture HTTP traffic using Wireshark or tshark without altering the evidence.
- Identify the SYN, SYN-ACK and ACK packets in a TCP handshake.
- Extract source/destination ports, sequence numbers, acknowledgement numbers, timestamps and HTTP headers.
- Explain why loopback captures do not show normal Ethernet MAC addresses and repeat the capture on a two-VM network where required.
- Generate a concise network forensic timeline and evidence report

TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- Md5sum and sha256sum
- Apache2
- Wireshark, tshark, Curl, Web browser

METHODOLOGY

First thing is to create a working environment. Created the folder structure before downloading or generating evidence of any form.

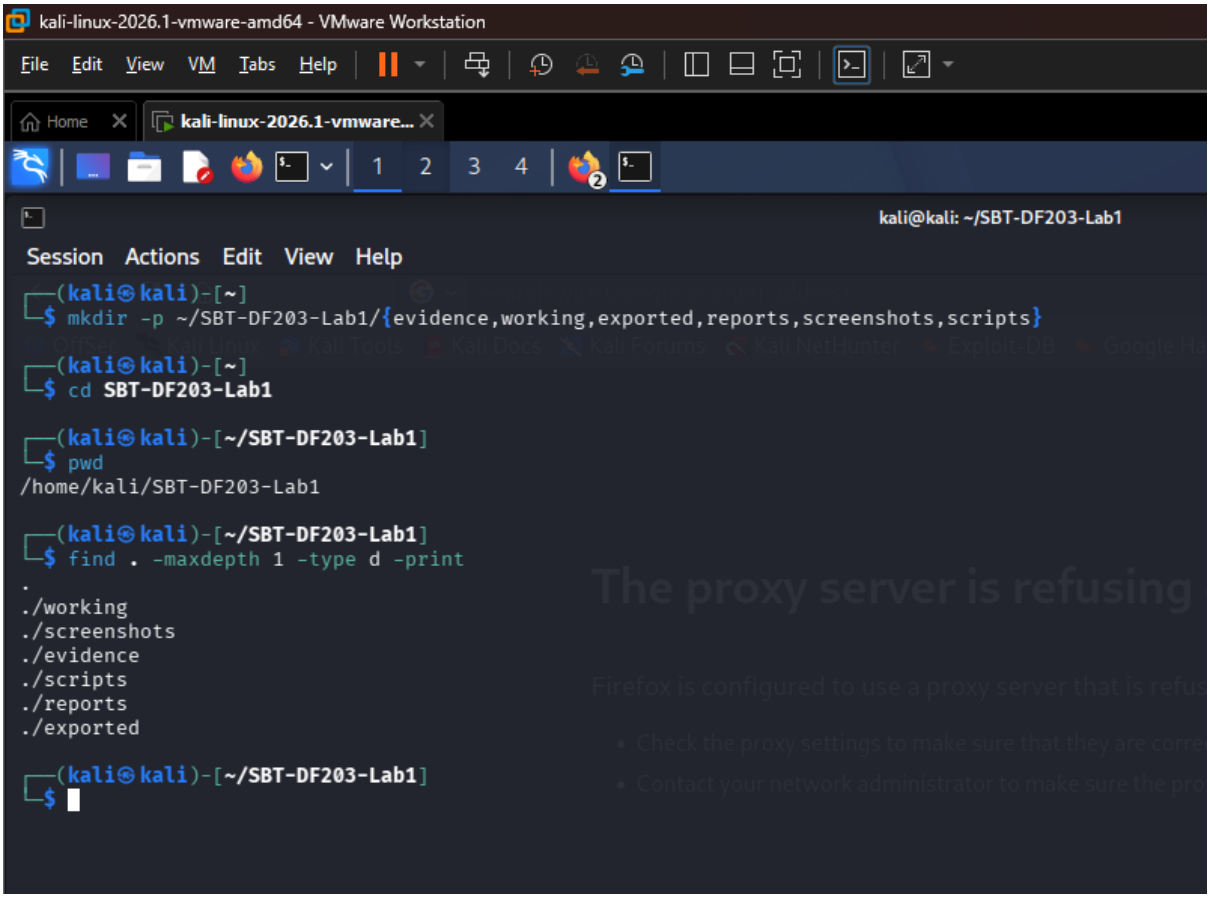

Fig 1: Working Environment created

Then we update the sudo environment by using the sudo apt update command and then we install the necessary toold needed to run the lab like, apache (the server tool), wireshark, tshark which are used for capturing live network traffic (or reads a saved capture file) and allows to inspect every packet in detail.

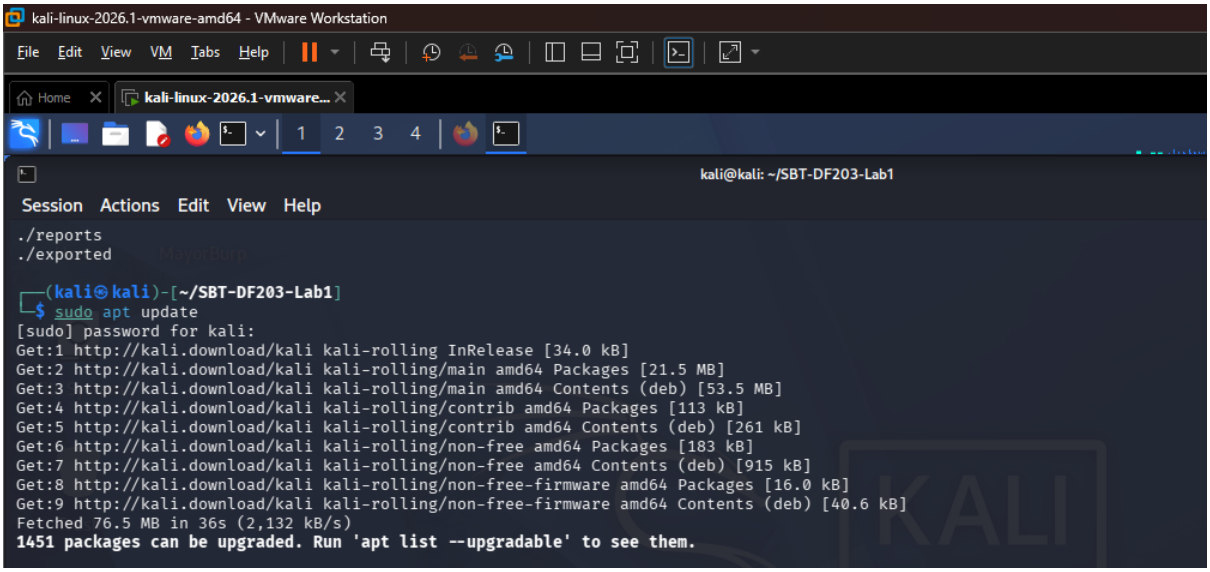

Fig 2: Updating the Environment Package

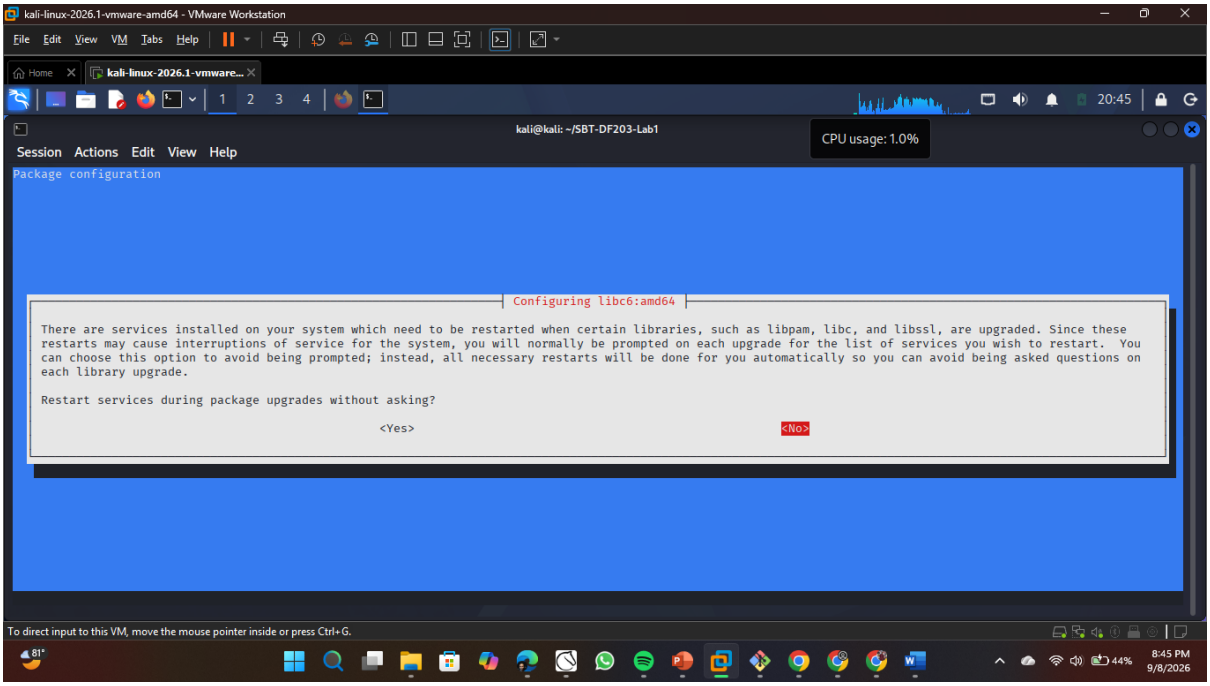

Fig 3: Picking yes for the configuration of Wireshark

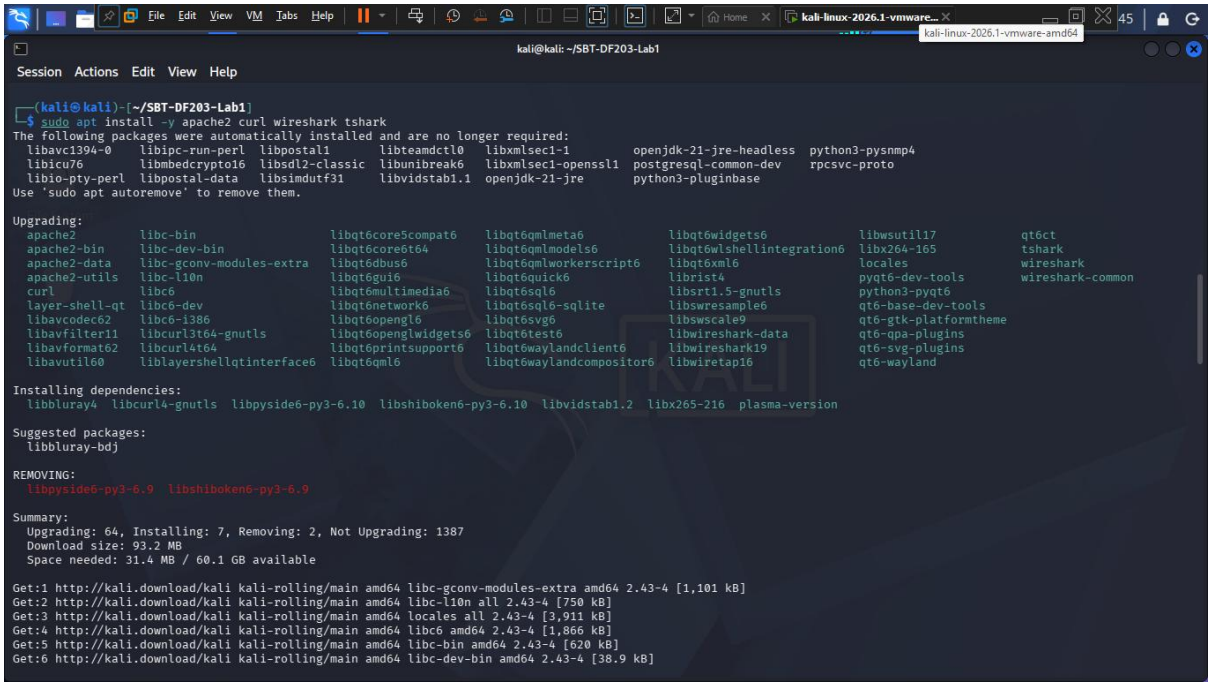

Fig 4: Picking yes for the configuration of Wireshark

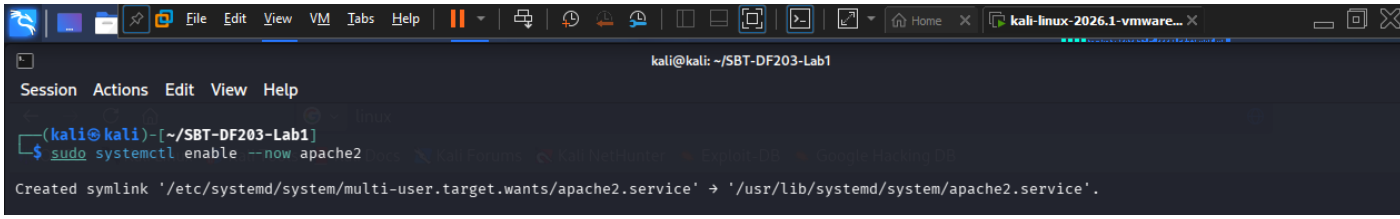

Fig 5: Enabling Apache

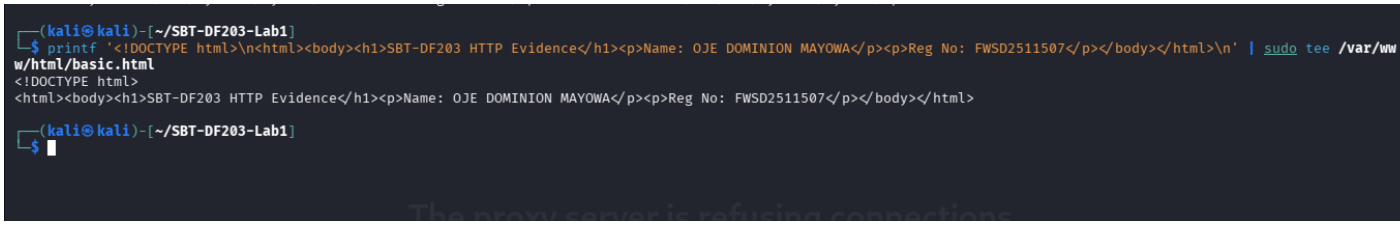

Fig 6: Creating a basic.html file using print f and using the tee command to put content into it

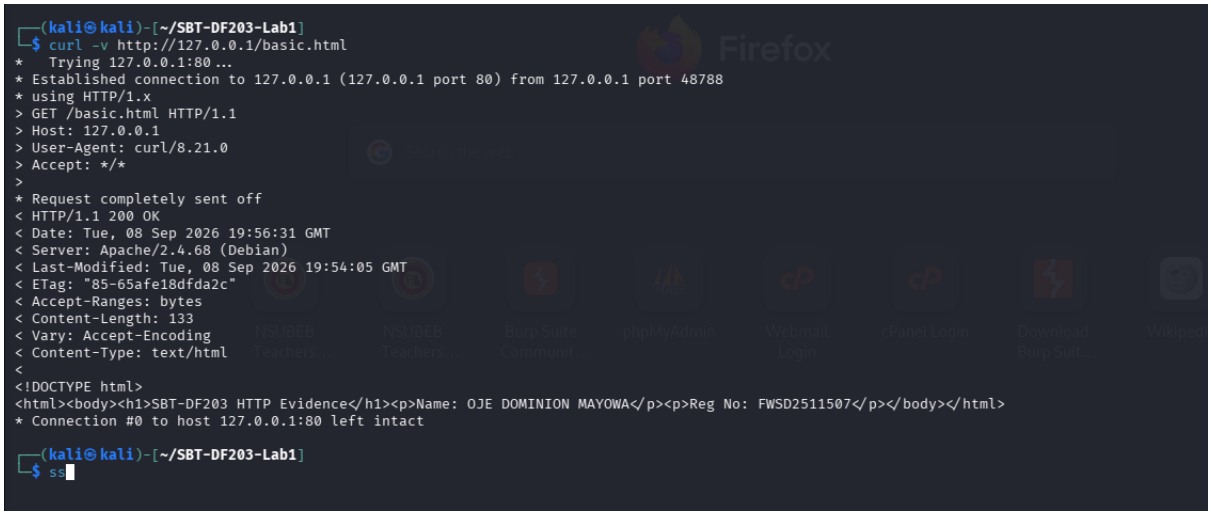

Fig 7: Using Curl to Establish connection with the browser

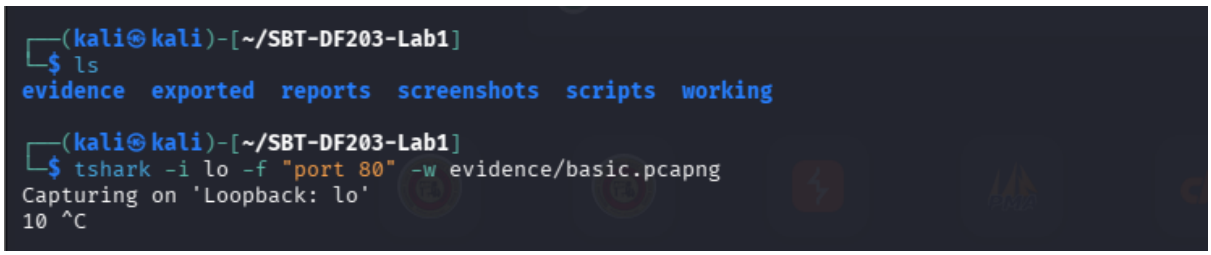

Fig 8: Opening a listener in the working environment using tshark

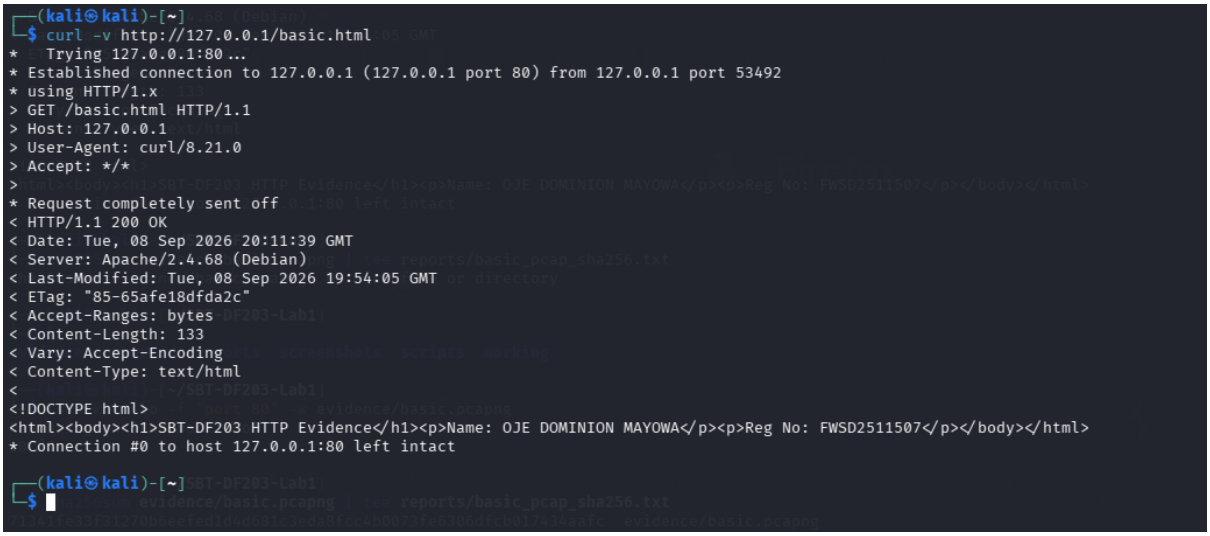

Fig 9: sending file packets by curling the basic web site

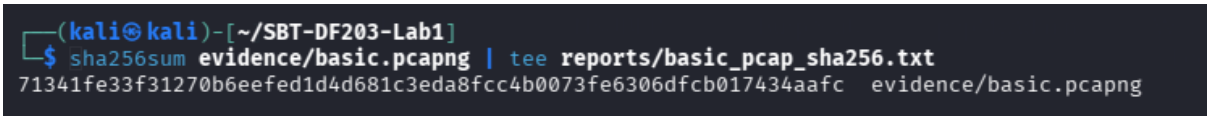

Fig 7: Hashing the Original Capture

Then to keep the Integrity of the Original evidence we create a copy of the evidence by using the cp command

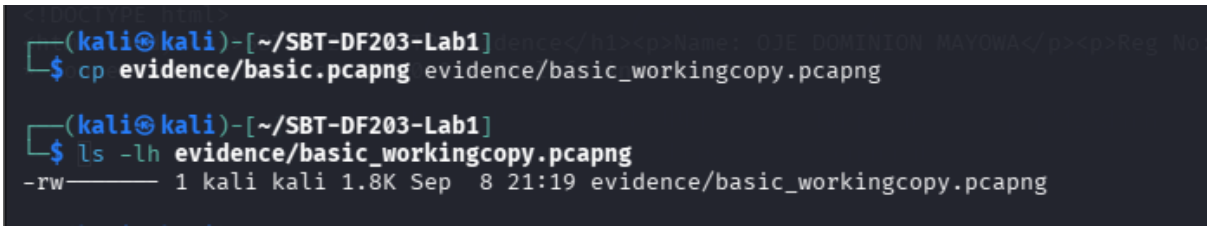

Fig 8: Creating a working copy of the Original Evidence

Then we also check the Integrity of the copy by hashing it also

Fig 8: Creating a working copy of the Original Evidence

From the figure above we see that the Integrity of the evidence is intact as the Hashed values of both the Original and Copy are the same thing.

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab1-OJE-DOMINION
Trainee name	OJE DOMINION
Date and time started	08/09/2026, 5:00pm WAT

Field	Student Entry
Evidence file name(s)	
Source or generation method	**tshark -i lo -f "port 80" -w evidence/basic.pcapng**
Original SHA-256	**71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc**
Working-copy SHA-256	**71341fe33f31270b6eefed1d4d681c3eda8fcc4b0073fe6306dfcb017434aafc**
Analysis workstation/VM	
Notes on any changes	

Part A - Verify the Web Service and Generate Text HTTP Traffic

The name and placeholders in the basic.html file have been replaced with my details

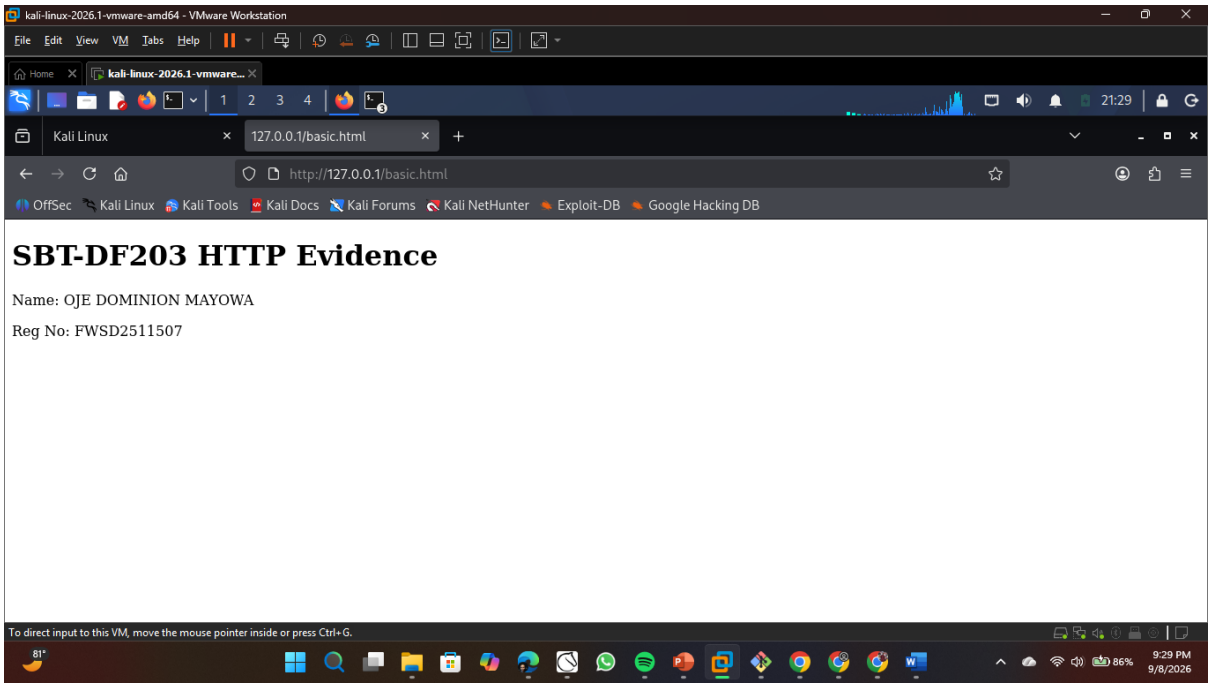

Fig 9: Basic.html file confirmed with necessary details

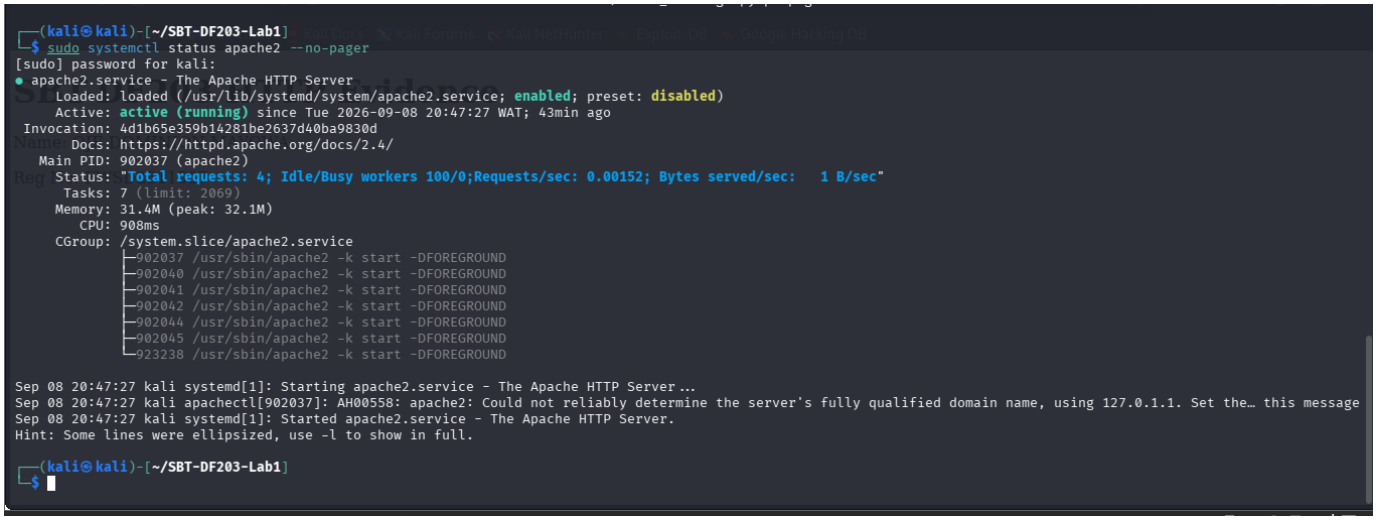

Fig 10: Apache Server Confirmed Active and running

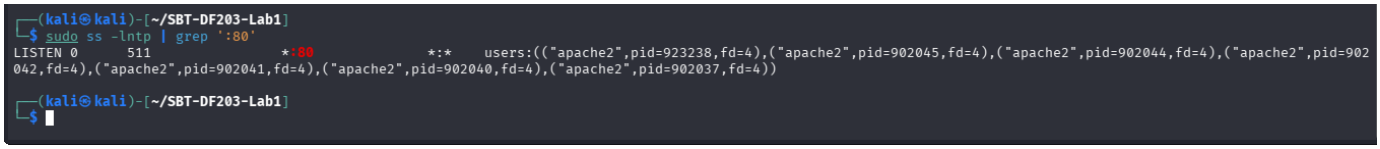

Fig 11: Confirms Apache is bound to Port 80 and is listening

The local service state is 511 and the listening port is port 80

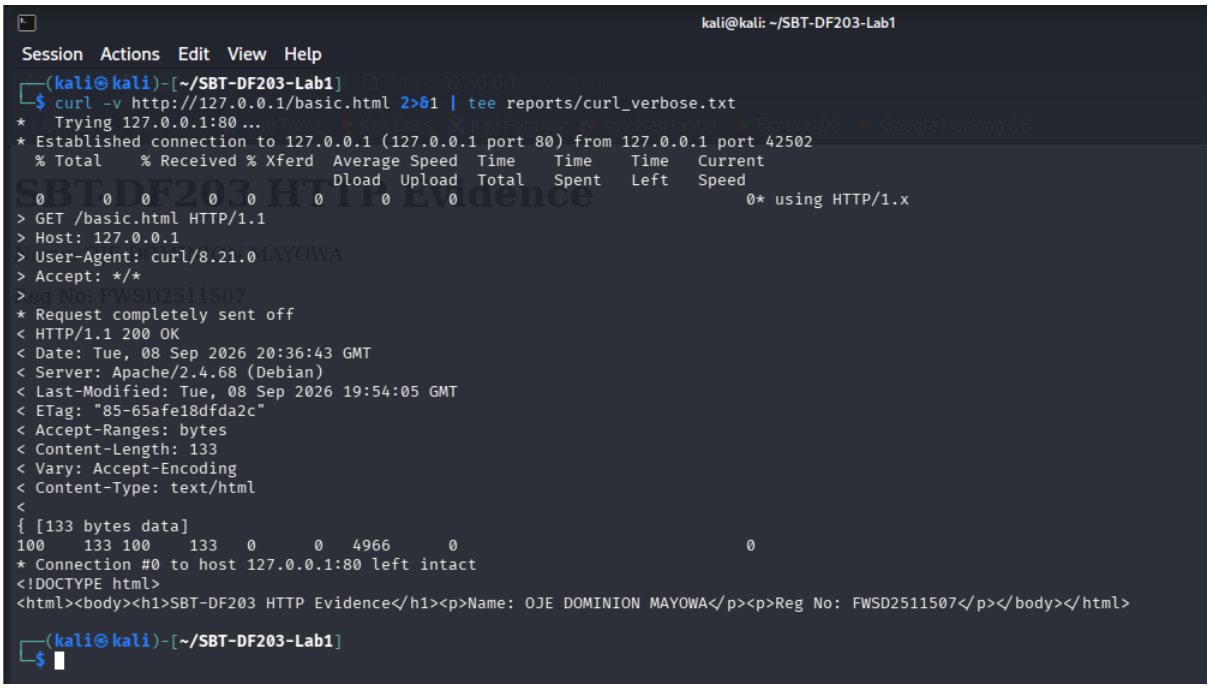

Fig 12 Using curl on the basic.html to get the response and request header

Now checking the saved response and request headers using the cat command on the curl_verbose.txt file where they were saved.

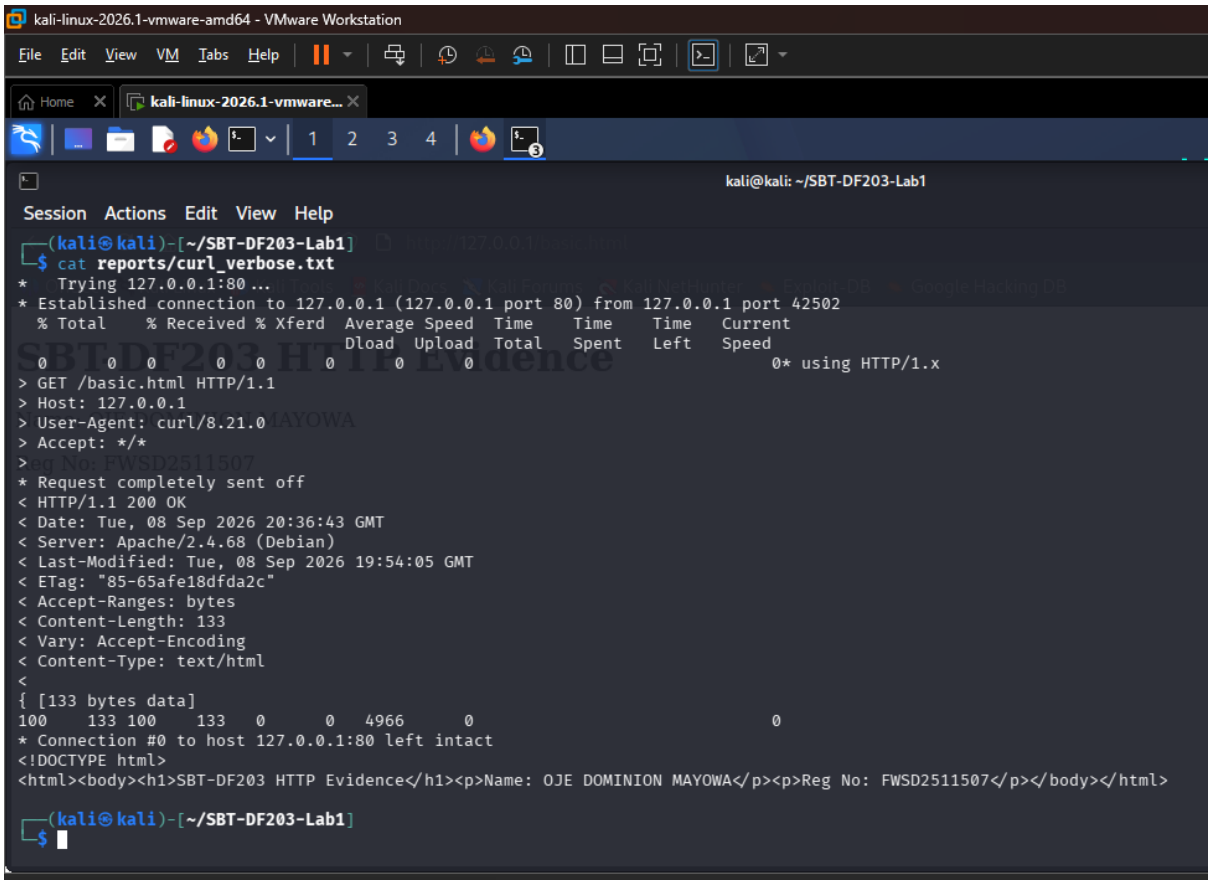

Fig 13: Confirms Apache is bound to Port 80 and is listening

Part B - Capture the Session

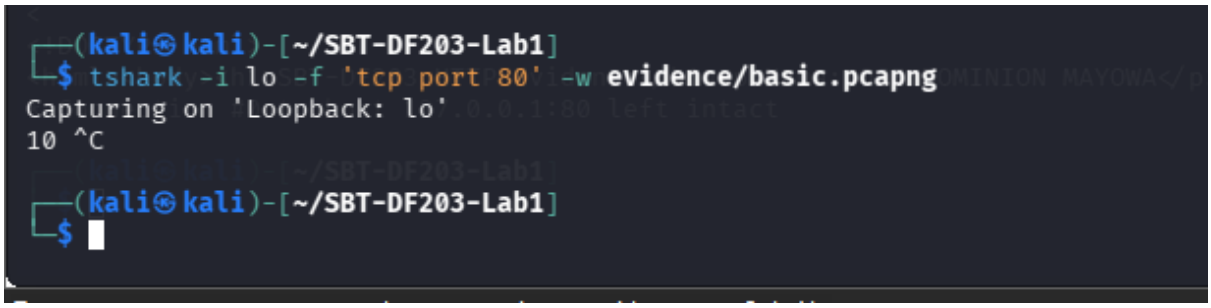

Fig 14: A Listening Port was opened using tshark

We then opened another terminal inside the same working directory to be able to request

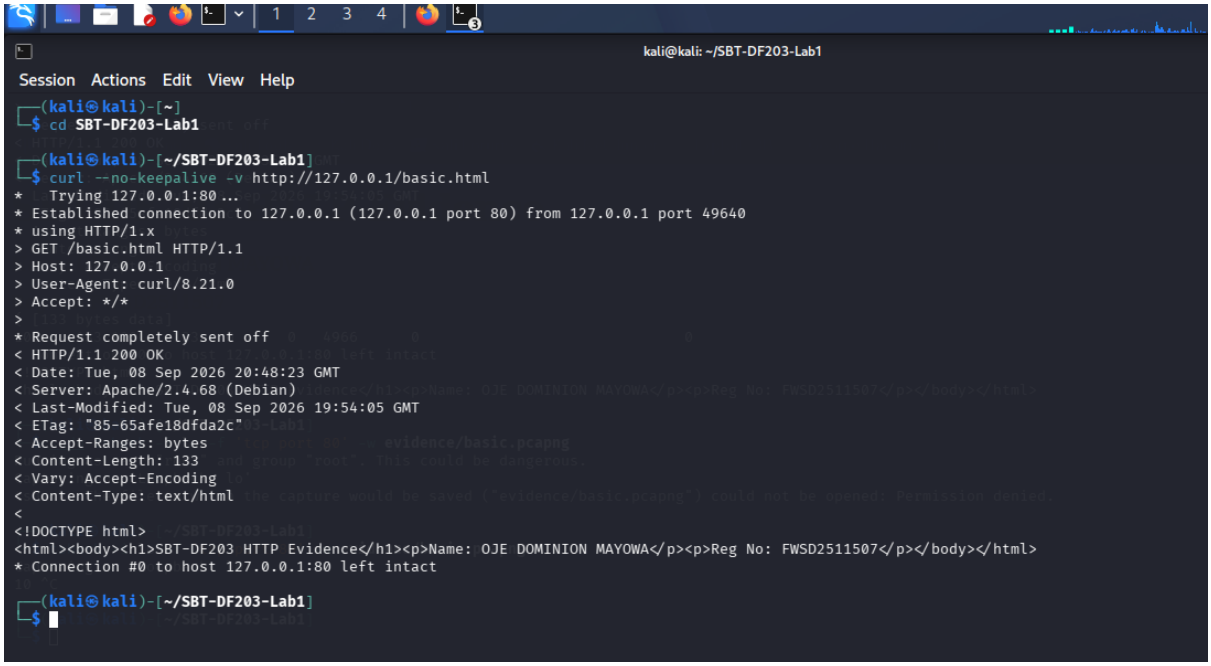

Fig 15: A Listening Port was opened using tshark

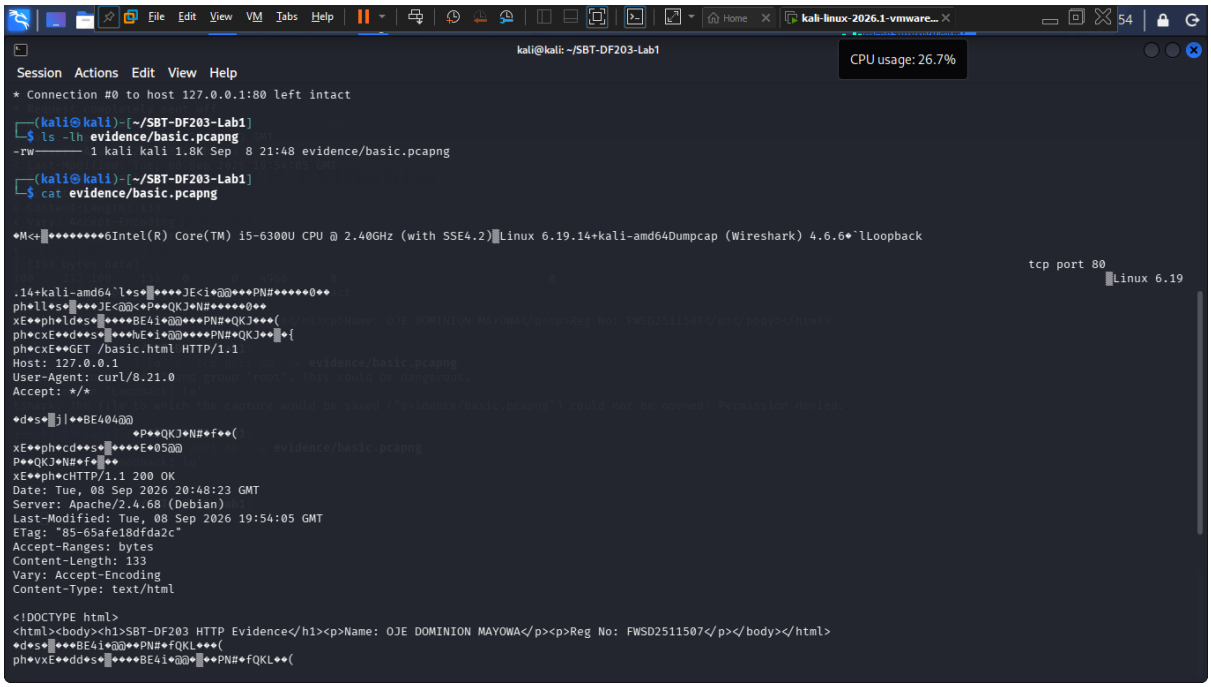

Fig 16: Confirmed the creation of the captured file

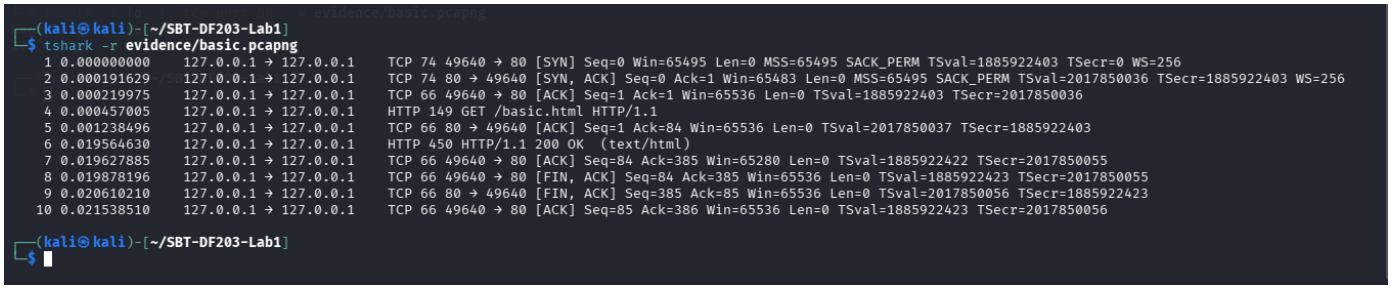

Fig 17: Confirmation of receiving packets

Then we create a copy of the original **basic.pcapng** to preserve the evidence while working on it

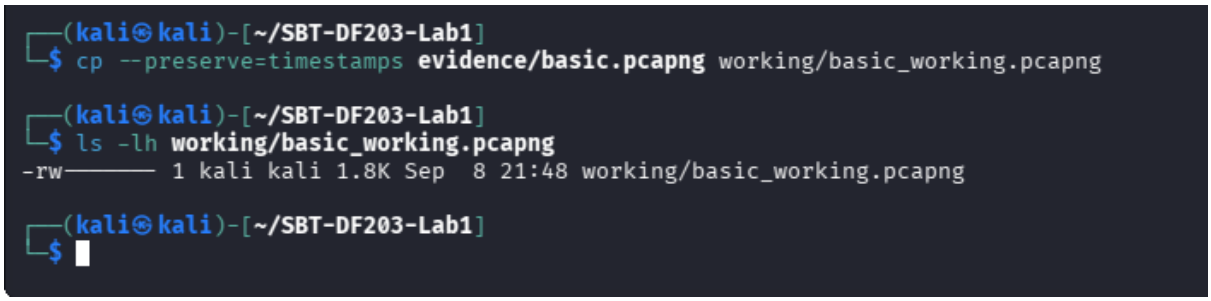

Fig 18: Creating a copy of the original evidence

Next, we hash it to verify the integrity of both the original evidence and the copy

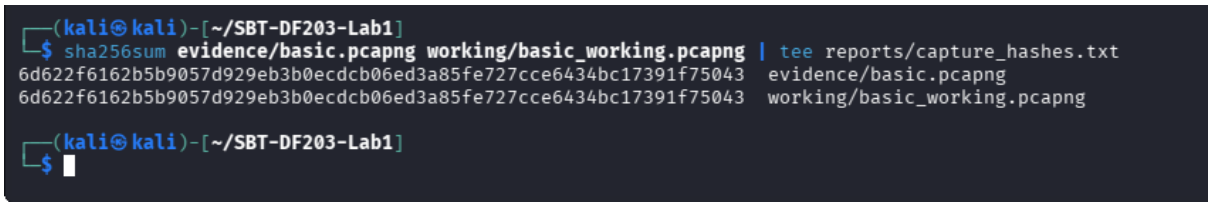

Fig 19: Checking the hash values of both the original and the copy

Part C - Analyze the TCP Three-Way Handshake

We run the tshark to be able to get the three handshake by analyzing the TCP for SYN, SYN-ACK and ACK.

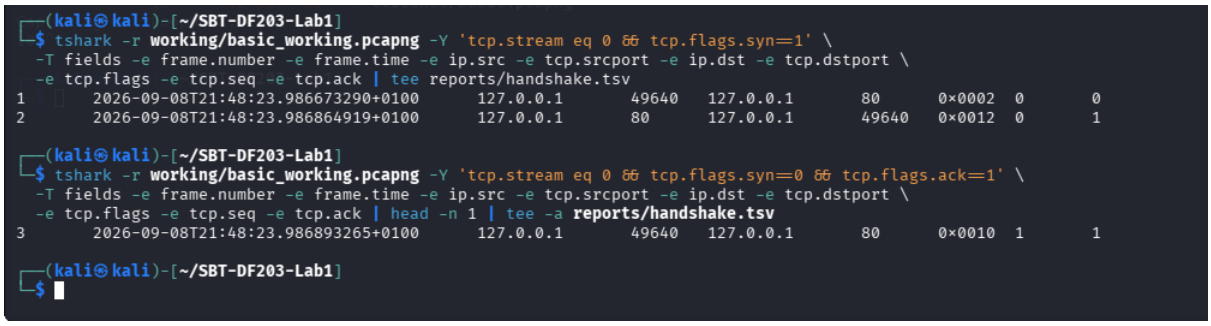

Fig 20: Checking the hash values of both the original and the copy

Packet	Flags	Client Seq	Server Seq/ACK	Timestamp	Interpretation
1	SYN	**Seq = 0**	**Ack = 0**	**21:48:23.986673290+0100**	Client requests a TCP session.
2	SYN, ACK	**Seq = 0**	**Ack = 1**	**21:48:23.986864919+0100**	Server acknowledges and provides its initial sequence number.
3	ACK	**Seq = 1**	**Ack = 1**	**21:48:23.986893265+0100**	Client acknowledges; the connection becomes established.

Part D - Examine the HTTP Request and Response

We then examine the HTTP Request and Response of the packets sent on the port

Fig 21: Examining the HTTP Request and Response

GET method = /basic.html
URI Host = 127.0.0.1
User-Agent = curl/8.21.0
Status Code = 200 / OK,
Server = Apache/2.4.68 (Debian),
Content - Type = text/html

Fig 22: Checking the Progression

Looking at the seq/ack numbers alongside the packet lengths, I could actually see how TCP keeps track of data as it moves between the client and server.

For the first three packets (the handshake: SYN, SYN-ACK, ACK), the length is 0 because no actual data is being sent yet, just the initial setup of the connection.

Then in packet 4, the client sends the HTTP GET request, which is 83 bytes long. Right after, in packet 5, I noticed the server's ack number was 84, which makes sense once I realized it's just the client's starting sequence number (1) plus the 83 bytes it just received. So the ack number is basically the server saying "I got everything up to byte 84."

The same pattern happens in the other direction: in packet 6, the server sends back its response (the HTML page), which is 384 bytes. In packet 7, the client's ack becomes 385, again matching the server's starting sequence (1) plus the 384 bytes sent.

What this told me is that ack equals previous seq plus previous length, every single time. That's how TCP confirms nothing was lost or corrupted along the way; every byte sent gets accounted for in the next acknowledgement.

The last few packets (8 to 10) are the connection closing down with FIN/ACK, and even those follow the same plus 1 pattern since a FIN counts as one byte for sequencing purposes.

Overall, this was a good way for me to actually see something I'd only read about before: how TCP's reliability really works at the byte level, not just in theory.

Part E - Inspect Encapsulation and Connection Closure

```
┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$

┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$ tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
  -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
8       2026-09-08T21:48:24.006551486+0100      127.0.0.1       49640   127.0.0.1       80      0×0011  84      385
9       2026-09-08T21:48:24.007283500+0100      127.0.0.1       80      127.0.0.1       49640   0×0011  385     85

┌──(kali㉿kali)-[~/SBT-DF203-Lab1]
└─$
```

Fig 23: Shows the ending and closing connection of the FIN/ACK Packets

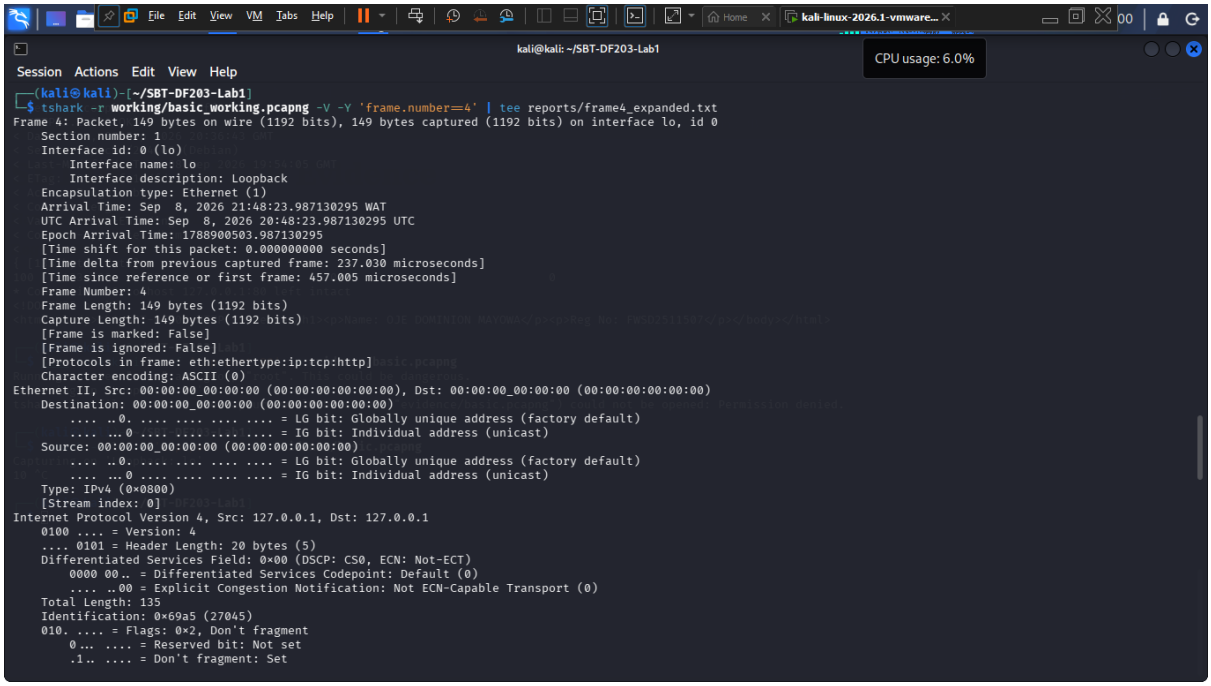

Fig 24: Listing out the Frame length, IP total Length, TCP header and TCP Payload

Que: Explain the relationship among frame length, IP total length, TCP header length and TCP payload length.

Each layer adds its own header on top of the payload from the layer above it. The Frame length includes everything (Ethernet + IP + TCP + HTTP data). If we remove the Ethernet header and you would get the IP total length. If we remove the IP header you get the TCP segment. If we remove the TCP header, we are left with just the actual HTTP data being carried. Every layer "wraps" the one above it.

Que: Locate FIN/ACK packets or a reset that closes the session. Explain the observed termination sequence.

Packet 8 carries flags 0x0011 (FIN, ACK), which is the server telling the client it has finished sending data and is ready to close its side of the connection, while also acknowledging the data it had already received.

Packet 9 also shows flags 0x0011 (FIN, ACK), this time from the client, acknowledging the server's FIN and sending its own FIN to close its side of the connection too.

Packet 10 shows flags 0x0010 (ACK only), which is the final acknowledgement confirming both sides agreed the connection is fully closed.

No RST packets were observed anywhere in the capture, which means the connection ended properly rather than being forcibly terminated, a strong indicator that the HTTP transaction completed successfully with no errors or unexpected drops.

Que:
Loopback (lo) traffic never touches an actual network interface card or physical medium, so there is no need for Ethernet framing or MAC addressing at all. Data goes directly through the kernel's network stack from one local socket to another, skipping the data-link layer entirely. That's why Wireshark shows something like 00:00:00_00:00:00 or no Ethernet header at all for loopback captures. To capture genuine Ethernet-layer info (real source/destination MAC addresses), you need traffic to actually cross a real or virtual NIC, which is why the lab asks you to repeat the capture between two separate VMs on a host-only network.

Required Forensic Findings

Finding	Value/Observation
Capture start and end time	Started 2026-09-08 21:48:23.986673290 (packet 1, SYN); It ended shortly after with the FIN/ACK teardown (packets 8–10), all within the same second, the total capture duration was under 1 millisecond
Client IP and port	127.0.0.1, port 49640
Server IP and port	127.0.0.1, port 80
Client initial sequence number	0 (relative sequence number as shown by tshark)
Server initial sequence number	0 (relative sequence number)
HTTP request timestamp	2026-09-08 21:48:23.987130295

Finding	Value/Observation
Requested URI	/basic.html
HTTP response code	200 OK
Content type and length	text/html, 133 bytes
Connection termination method	good termination via FIN/ACK exchange (packets 8–9 showed flags 0x0011, confirming FIN, ACK), not an abrupt RST
MAC-address observation	Source and destination MAC addresses both showed as 00:00:00_00:00:00 — expected on loopback, since traffic never leaves the kernel to touch a real network interface, so no genuine Ethernet framing/MAC addressing occurs

This lab gave me hands-on insight into how HTTP traffic is structured, transmitted, and closed at the packet level, and how each protocol layer builds on the one before it. More importantly, it introduced me to real forensic practice, capturing evidence carefully, verifying its integrity through hashing, and documenting findings in a professional way These skills are foundational to a good digital forensics trainee or expert.